



Instituto Superior de
Engenharia do Porto

Relatório técnico - 2ª Iteração

Helder Sousa Nº1231982

Rui Pinto Nº1230906

Rodrigo Martins Nº1241122

Introdução	2
Design	3
Design: Arquitetura Física	3
Design: Arquitetura Lógica	3
Design: Organização do Código	4
Class Diagrams	4
Design: model Class Diagram.....	5
Controller Class Diagram	6
Global Class Diagram	7
Service Class Diagram	7
Views Class Diagram	8
Sequence Diagrams.....	8
Use Case 1 – Registrar Animal	8
Use Case 2 – Registrar Veterinário.....	9
Use Case 3 – Definir Serviço	9
Use Case 4 – Marcar Consulta	10
Use Case 5 – Consultar Marcações.....	10
Use Case 6 – Consultar Histórico de Serviços	11
Use Case 7 – Consultar Serviços de um Veterinário.....	11
Use Case 8 – Emitir Prescrição	12
Use Case 9 – Consultar Prescrições	12
Use Case 10 – Consultar Serviços Realizados	13
Implementação	13
Conclusão	14

Introdução

Na segunda iteração do projeto desenvolvido em Fundamentos de Desenvolvimento de Software, foi dado seguimento à construção de uma aplicação de linha de comanos chamada VeterinaryApplication, utilizando a linguagem de programação C++, com foco na Programação Orientada a Objetos, com ênfase na aplicação da estrutura Model–View–Controller (ou MVC) com a componente Service e Repository.

Objetivos Principais da Iteração

- Estruturar a aplicação com base na arquitetura MVC.
- Desenvolver os diagramas de classes
- Desenvolver os diagramas de sequência
- Aplicar os conceitos da programação orientada a objetos.

A aplicação utiliza uma arquitetura MVC com componente Service e também com Repository

Model: Dados e lógica de negócio

View: Interface e apresentação ao utilizador

Controller: Lógica da aplicação e coordenação

Service: Faz a ligação entre o Controller e o Model e verifica regras de negócio e validações

Repository: Responsável pela persistência e armazenamento de dados (neste caso em memória)

DTO (Data Transfer Object): Utilizado para transportar dados entre os componentes da aplicação, nomeadamente Views, Controllers e Services, reduzindo o acoplamento entre camadas.

Mapper: Converte objetos Model em objetos DTO e vice-versa, simplificando a transformação de dados entre camadas.

Vantagens desta arquitetura:

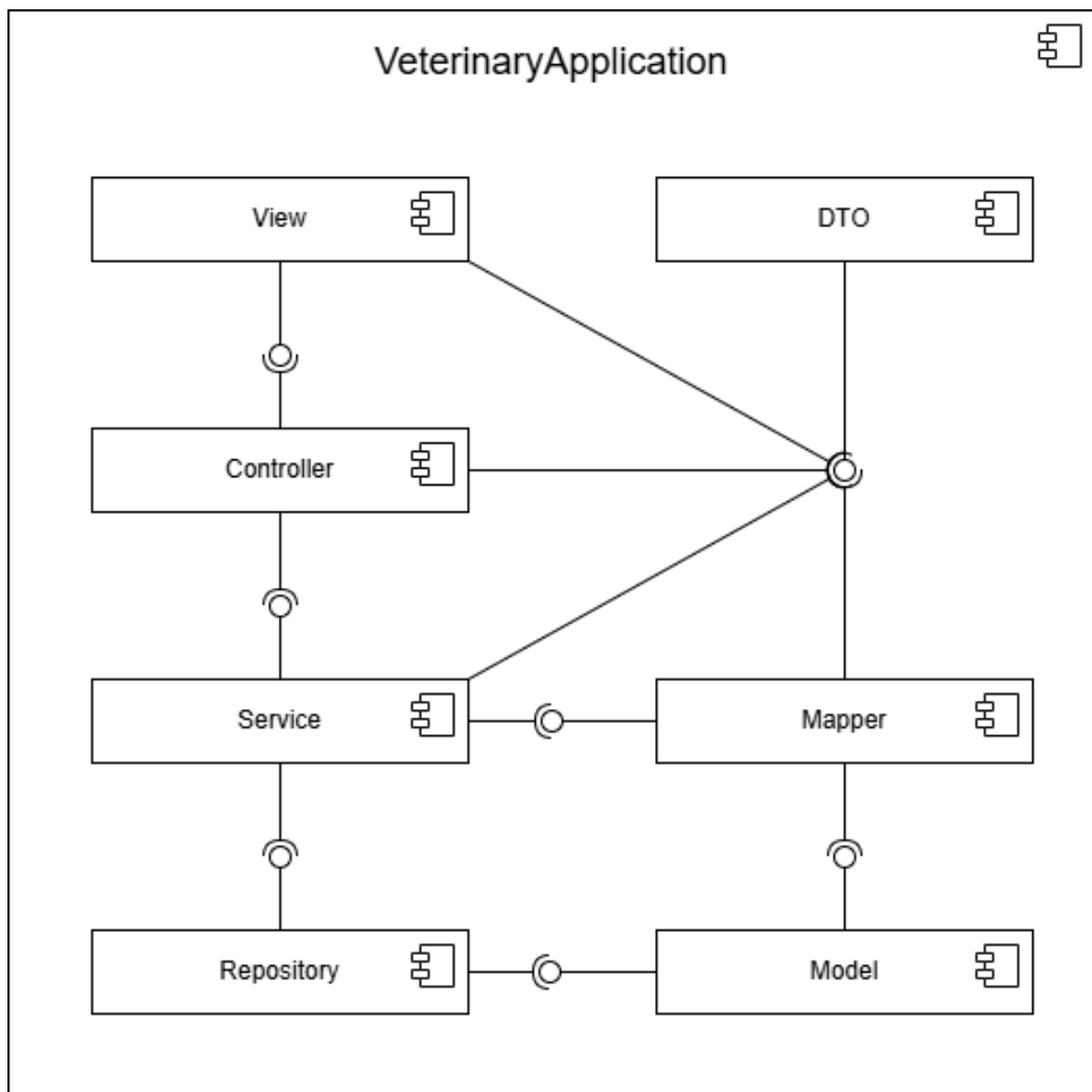
- Separação de responsabilidades
- Maior facilidade de manutenção e escalabilidade
- Melhor organização do código
- Melhor legibilidade e reutilização
- Maior facilidade de testes e depuração
- Suporte ao desenvolvimento em equipa e modularidade

Design

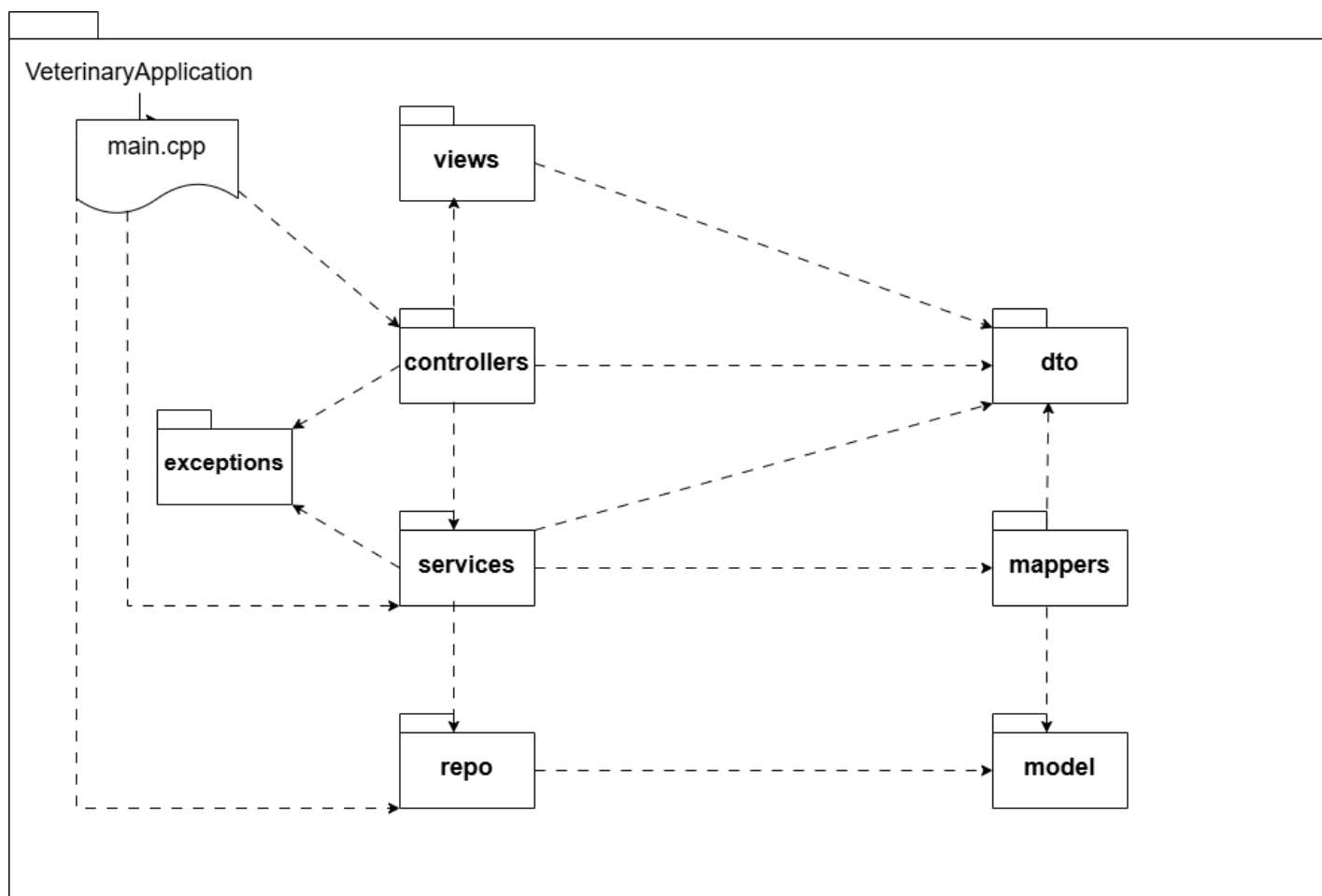
Design: Arquitetura Física



Design: Arquitetura Lógica

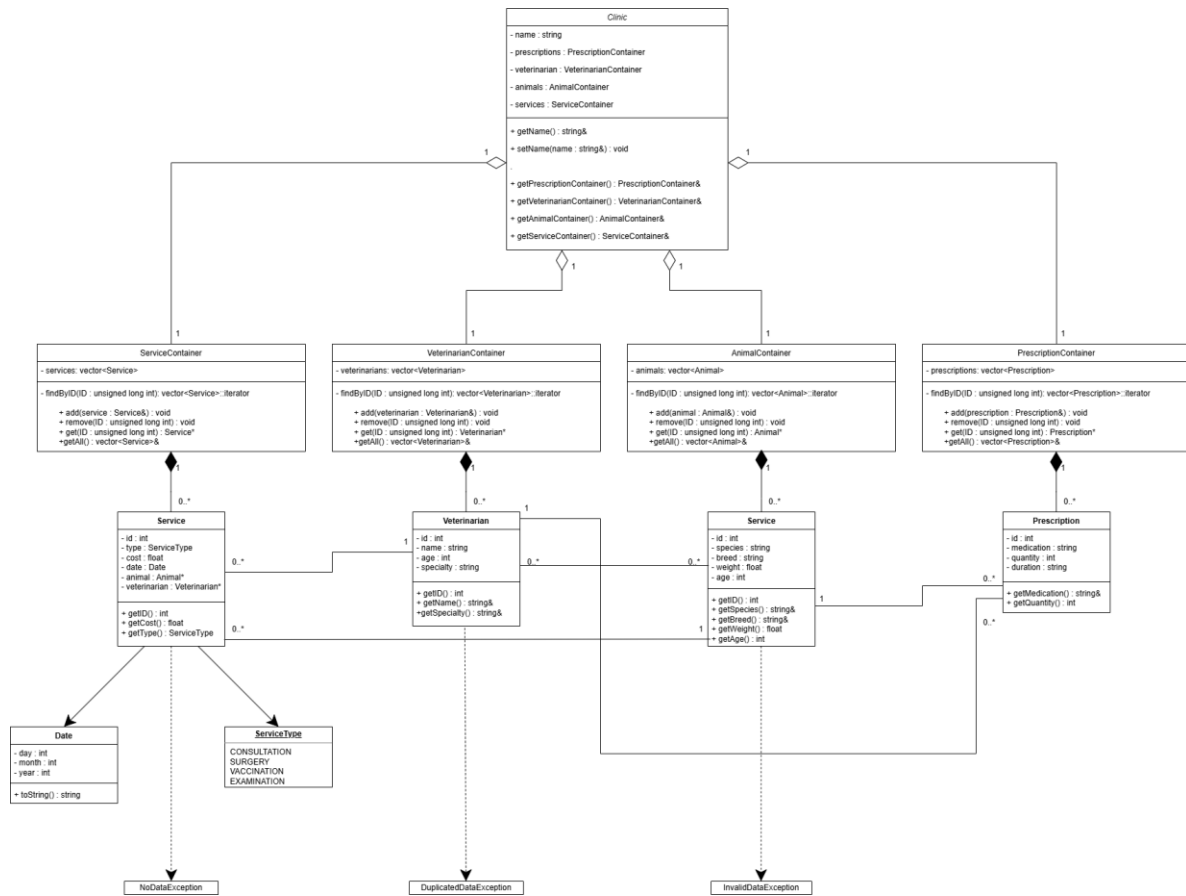


Design: Organização do Código

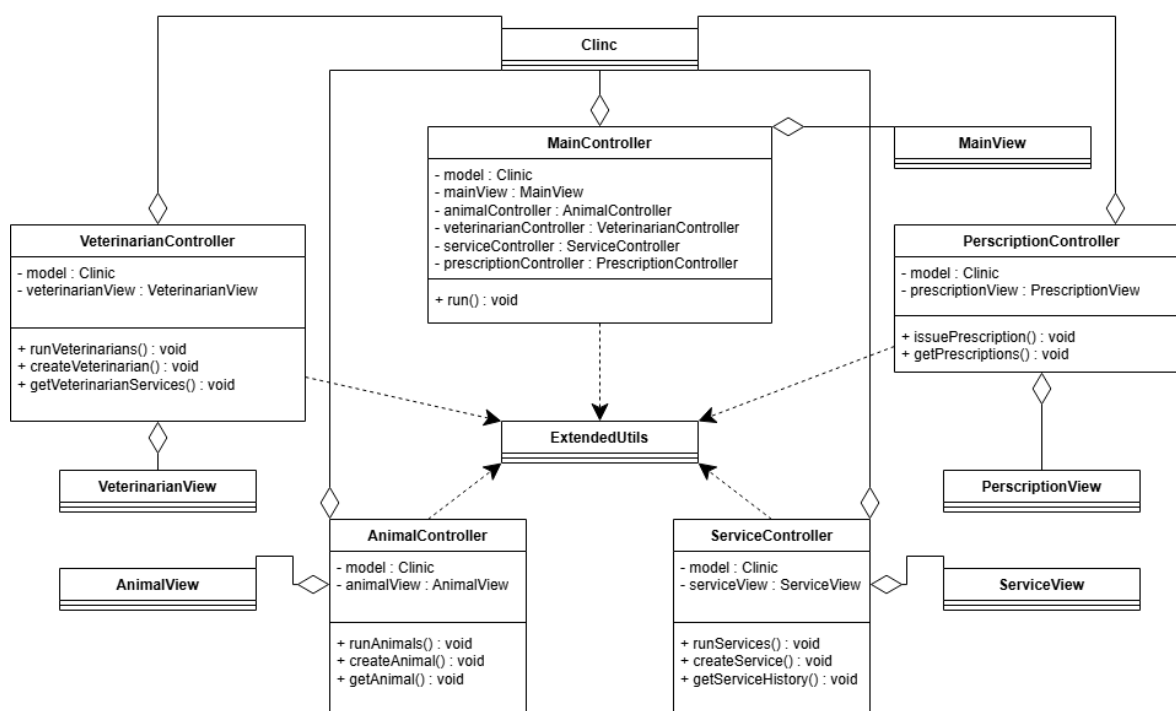


Class Diagrams

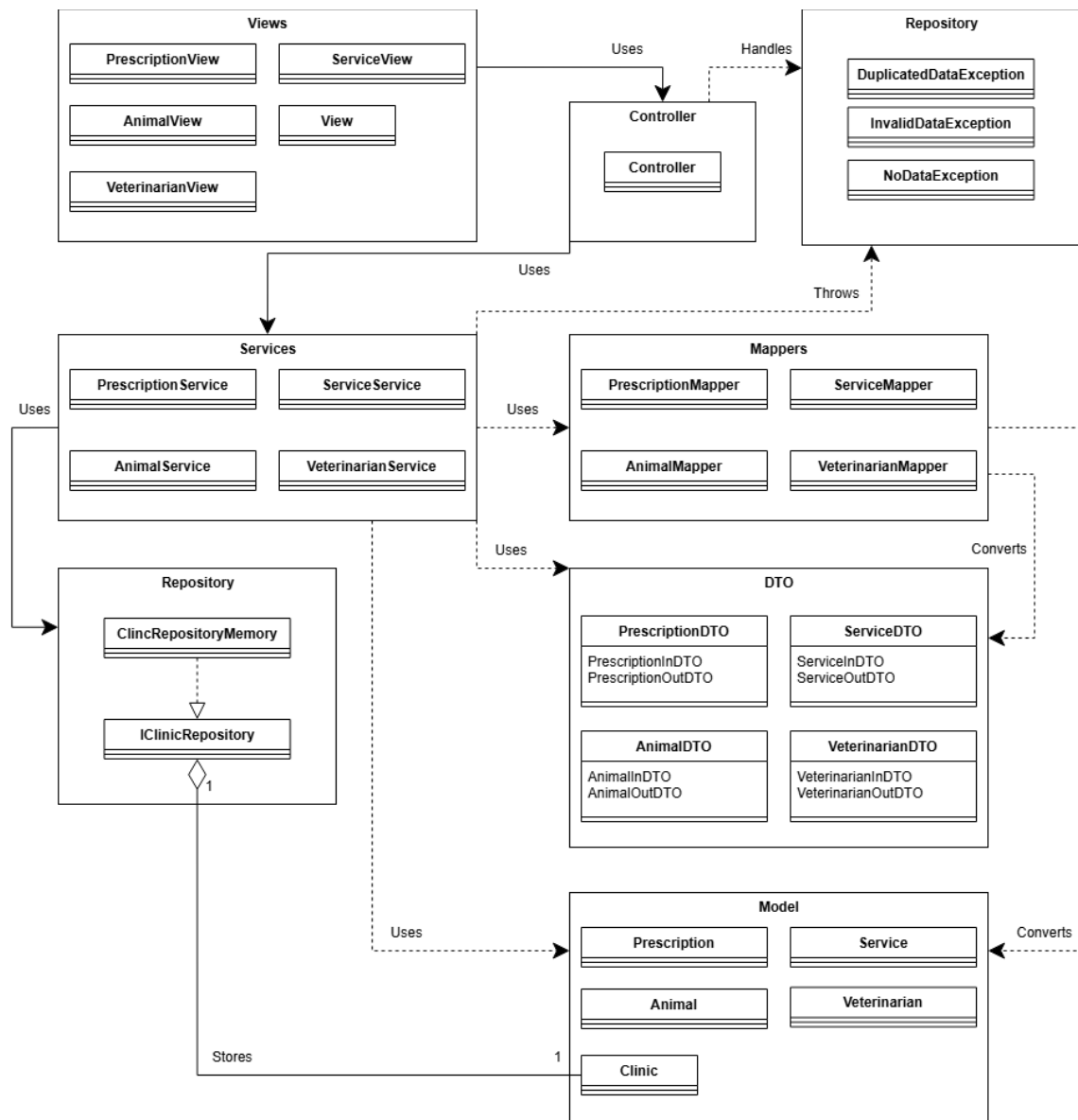
Design: model Class Diagram



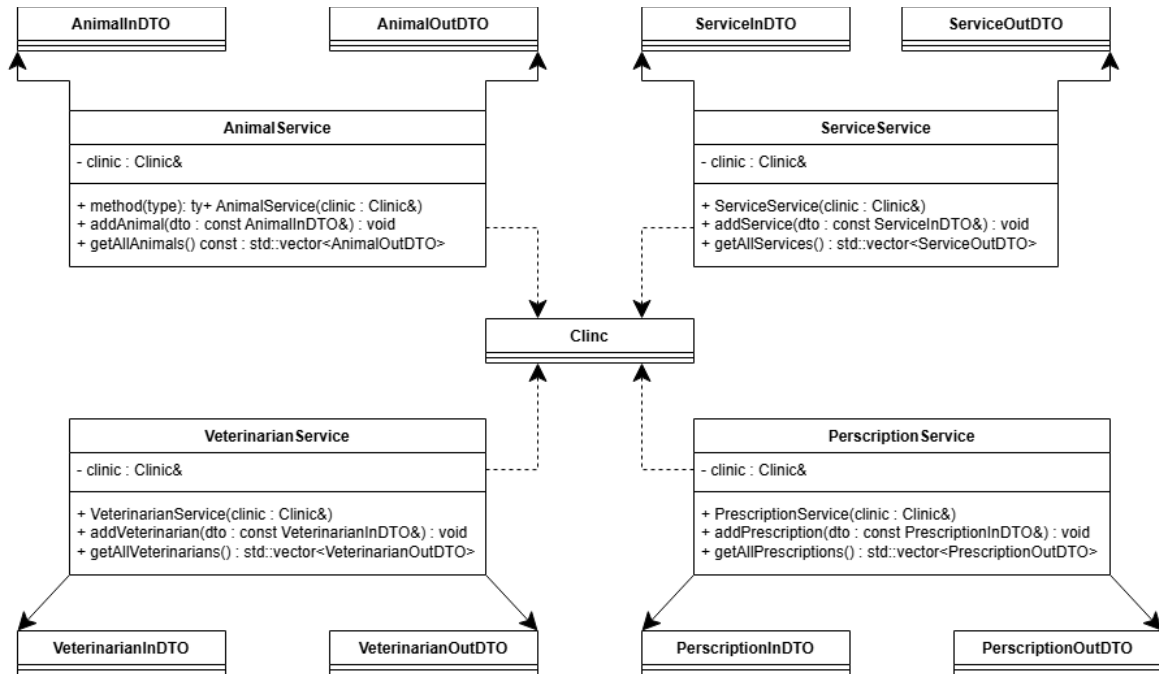
Controller Class Diagram



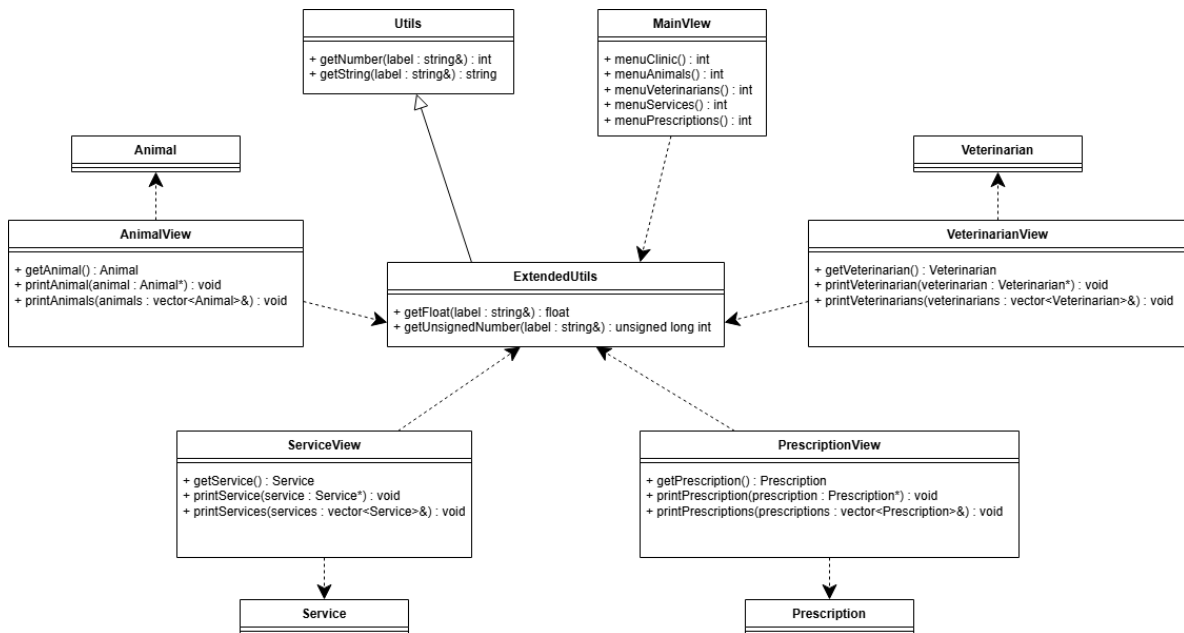
Global Class Diagram



Service Class Diagram

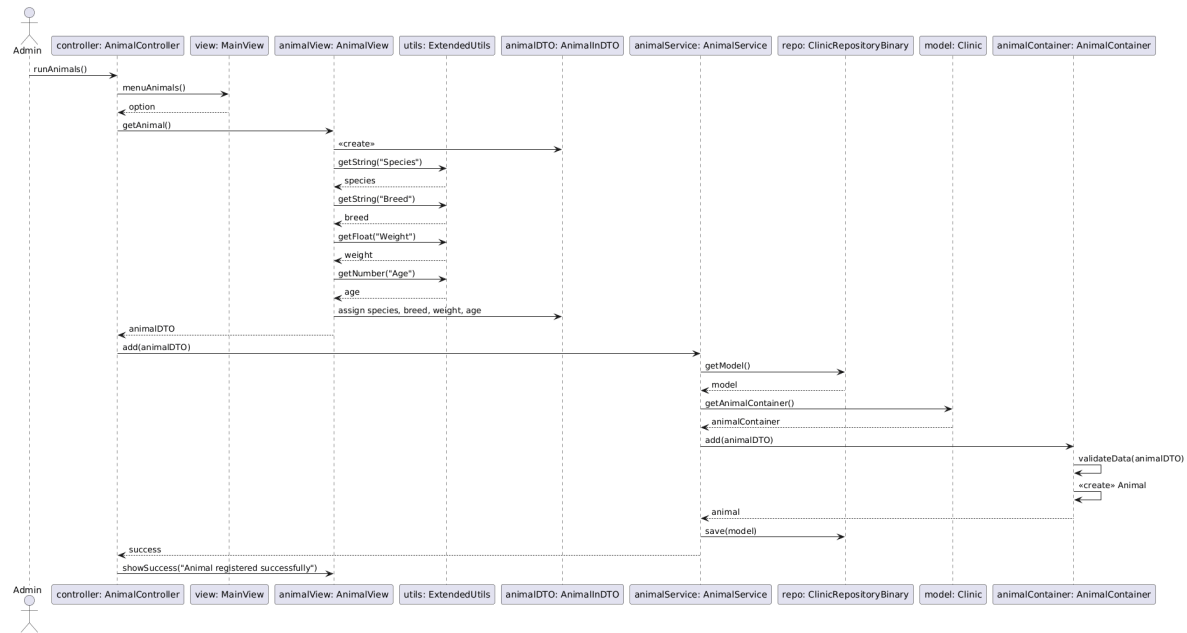


Views Class Diagram

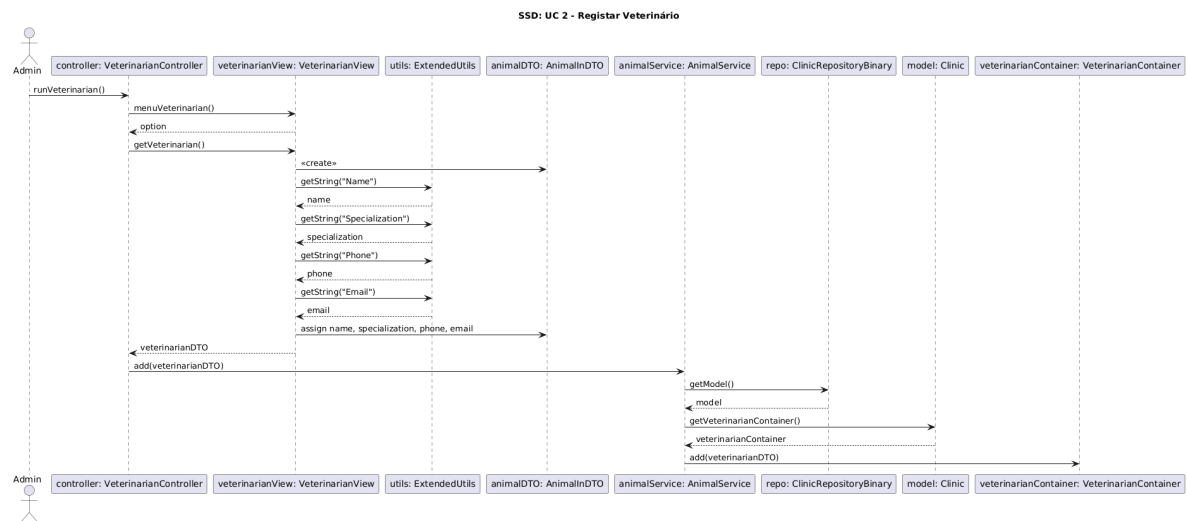


Sequence Diagrams

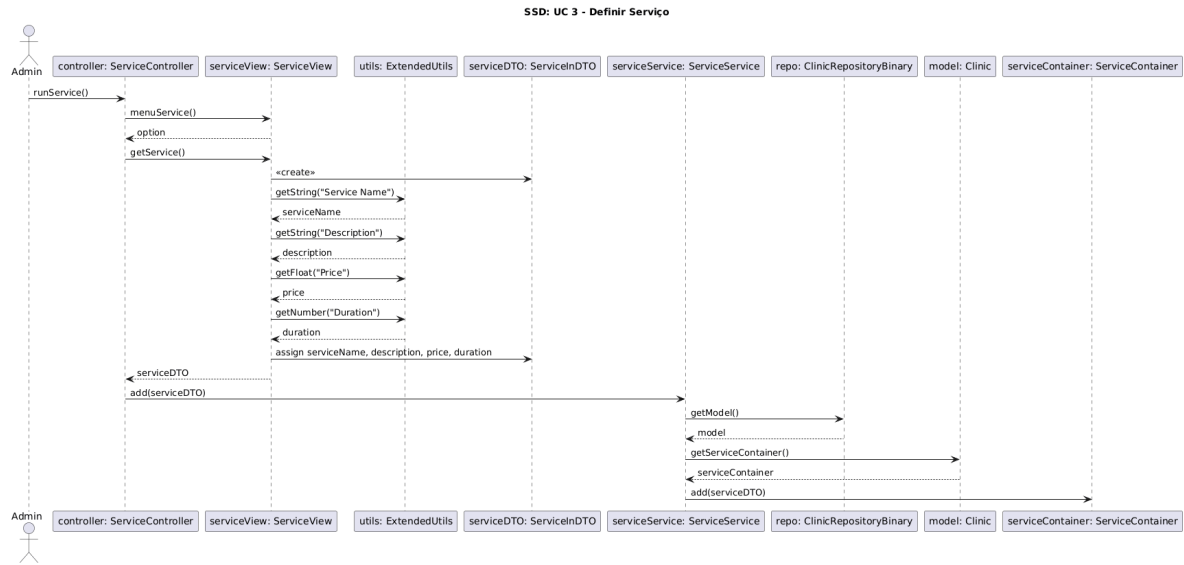
Use Case 1 – Registrar Animal



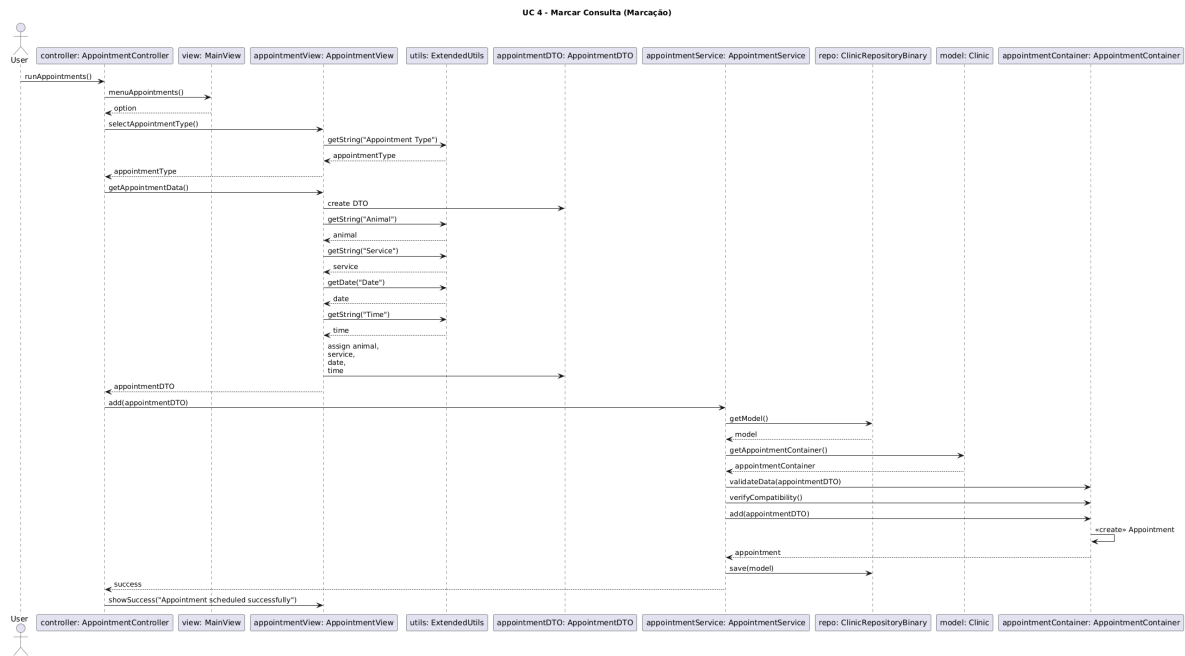
Use Case 2 – Registrar Veterinário



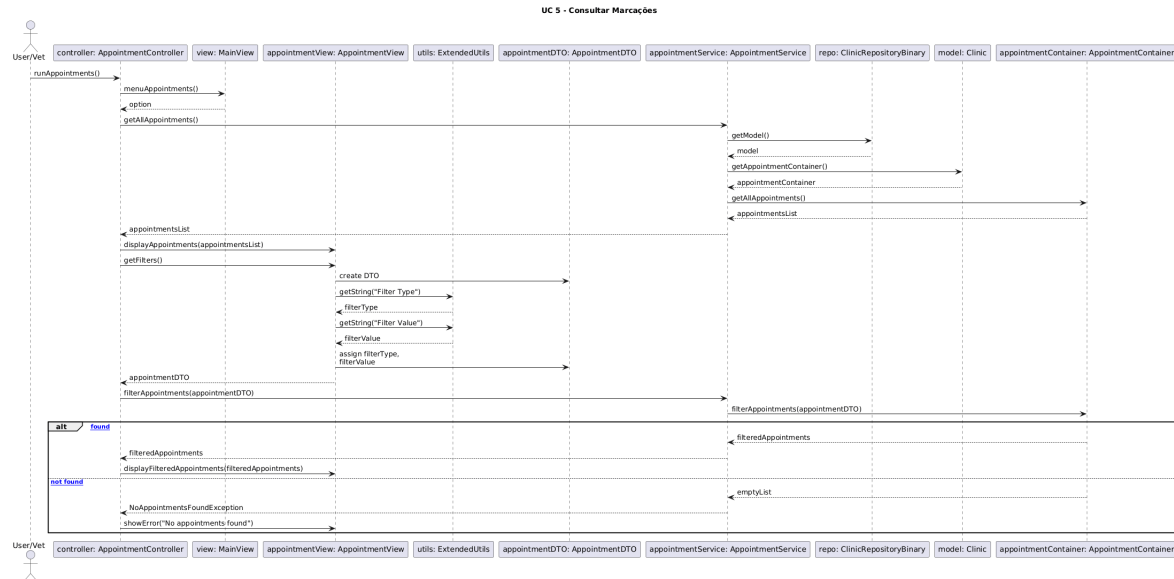
Use Case 3 – Definir Serviço



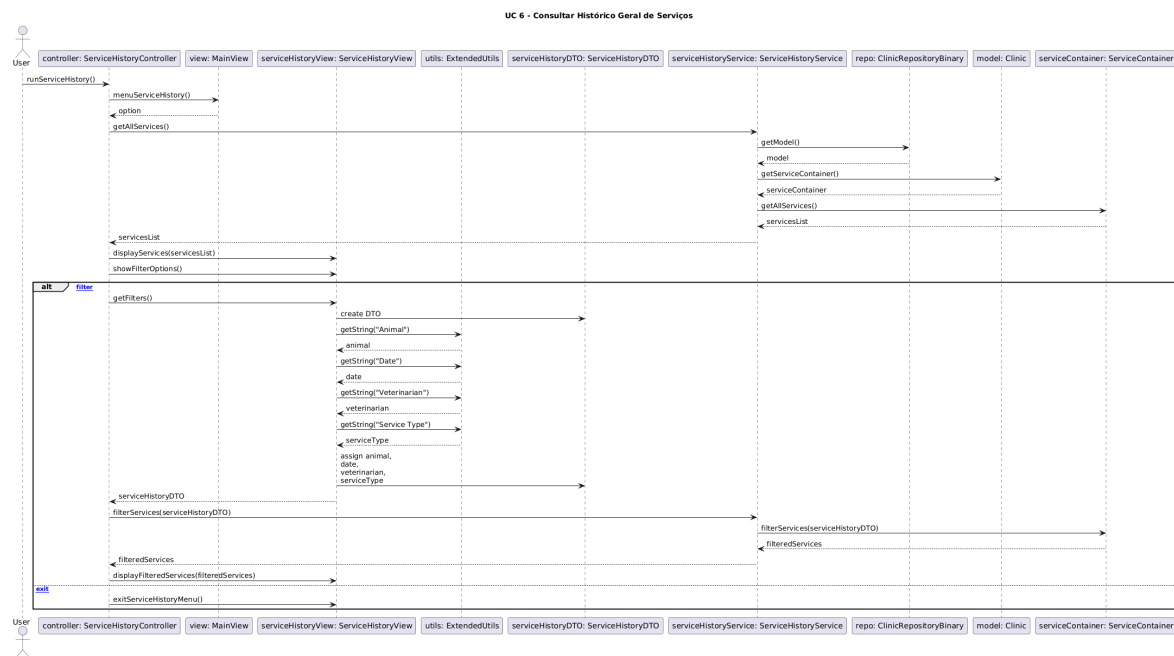
Use Case 4 – Marcar Consulta



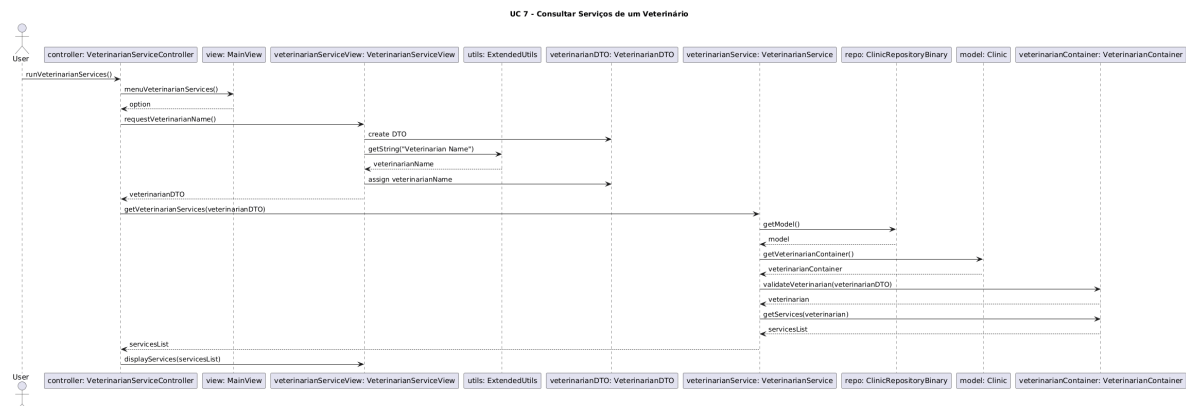
Use Case 5 – Consultar Marcações



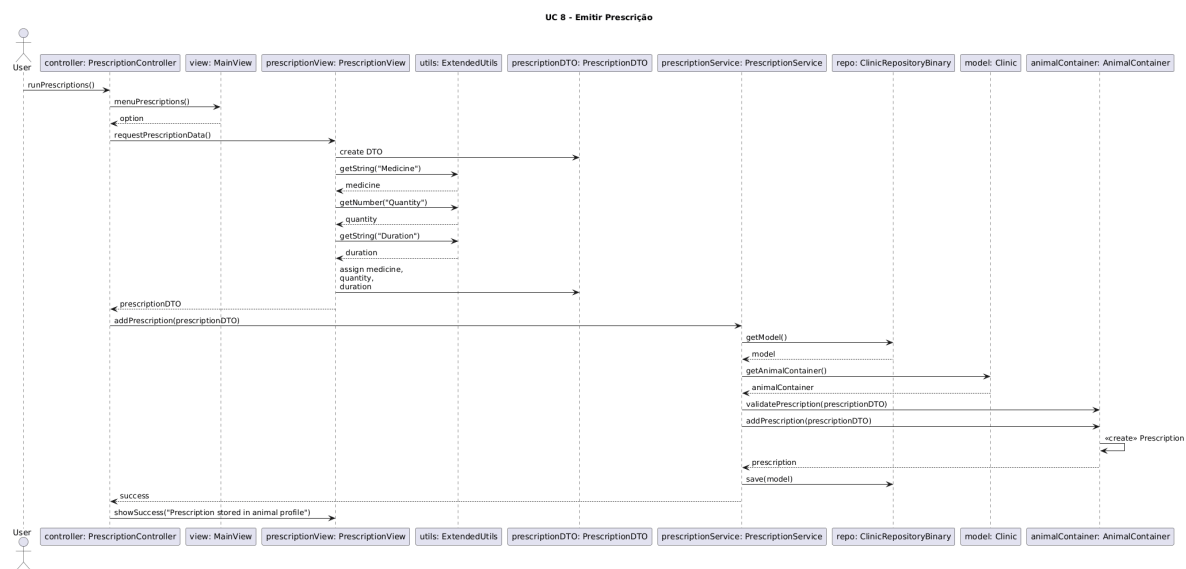
Use Case 6 – Consultar Histórico de Serviços



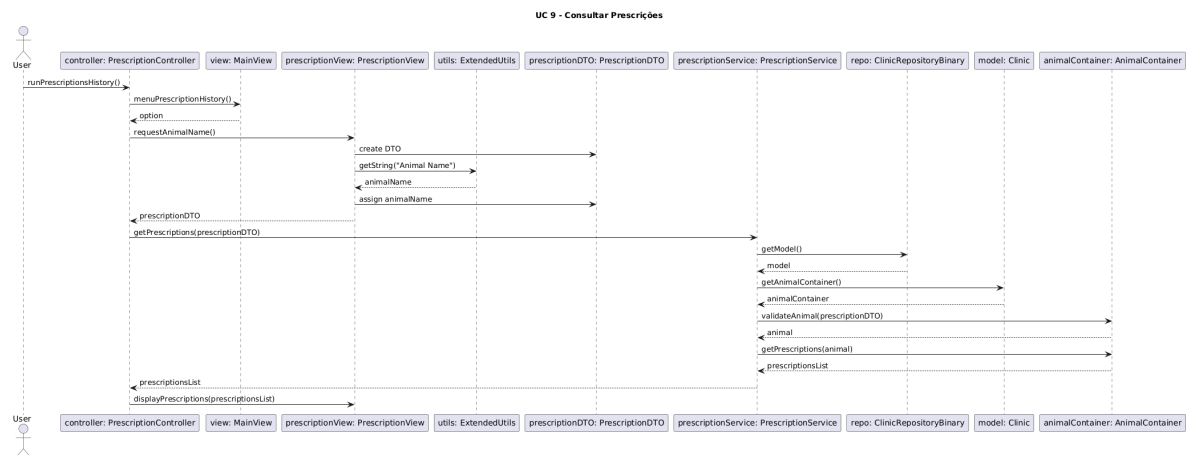
Use Case 7 – Consultar Serviços de um Veterinário



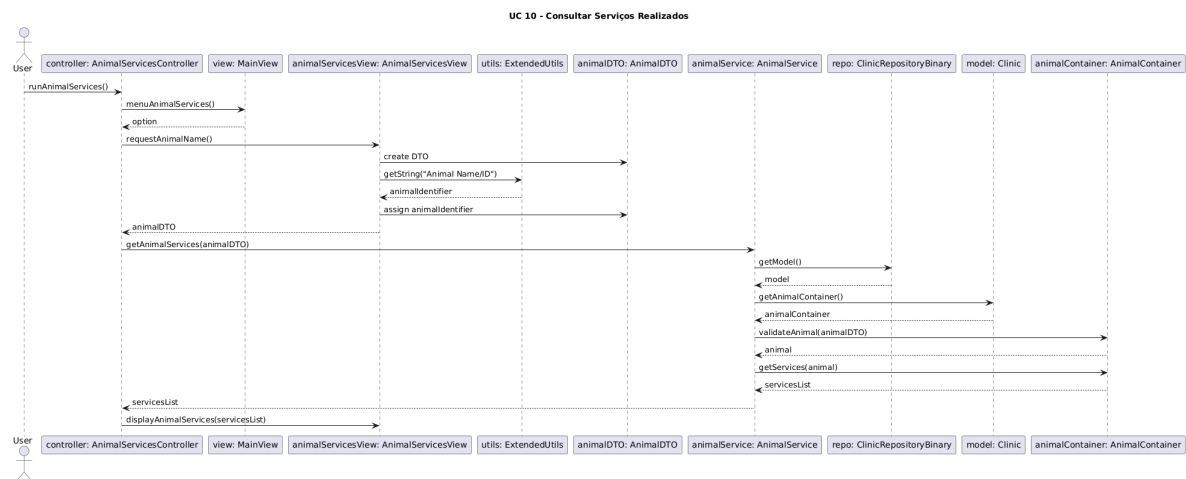
Use Case 8 – Emitir Prescrição



Use Case 9 – Consultar Prescrições



Use Case 10 – Consultar Serviços Realizados



Implementação

Estado da implementação

- Todos os menus estão funcionais
- Funções como:
 - .1. Adicionar animais e veterinários
 - .2. Listar animais e veterinários
 - .3. Prescrever medicamentos
 - .4. Marcar serviço
 - .5. Listar serviços e marcações
 - .6. Procurar animal por ID
- Controller implementado

Próximos objetivos

- Adicionar alguns ficheiros necessários ao repositório
- Melhorias estéticas e de qualidade de utilização
- Implementar as funcionalidades de use cases em falta
- Implementar Repositorio e persistência em ficheiros binários
- Implementar exceptions base
- Aplicar exceptions nos containers e services
- Adicionar validações simples aos dados

Conclusão

Nesta segunda depois de termos definido uma estrutura para como iríamos fazer o trabalho fomos construindo os diagramas e fazendo um 'esqueleto' do código, que fomos alterando de acordo com o desenvolvimento de cada diagrama, depois de termos os diagramas feitos e com a estrutura definida começamos a trabalhar nas implementações. Os diagramas criados iram ser de grande ajuda nas implementação das funcionalidades em falta.